

Contents

Lập Trình cơ bản.....	2
BÀI 1. ƯỚC SỐ CHUNG LỚN NHẤT VÀ BỘI SỐ CHUNG NHỎ NHẤT	2
BÀI 2. BẮT ĐẦU VÀ KẾT THÚC	3
BÀI 3. PHÉP CỘNG.....	3
BÀI 4. SỐ TĂNG GIẢM	4
BÀI 5. LIỆT KÊ VÀ ĐẾM	4
BÀI 6. SỐ ĐẦU TIÊN BỊ LẶP.....	5
BÀI 7. GHÉP HÌNH	6
BÀI 8. CHIA HẾT CHO 2.....	7
BÀI 9. ĐÔI TIỀN	8
BÀI 10. NHẦM CHỮ SỐ.....	8
BÀI 11. TỔNG NHỎ NHẤT	10
BÀI 12. SỐ KHỐI LẬP PHƯƠNG.....	10
BÀI 13. SỐ NHỎ NHẤT	11
BÀI 14. PHÂN SỐ ĐƠN VỊ	12
BÀI 15. SẮP XẾP XEN KẼ.....	13
BÀI 16. SẮP XẾP THEO KHOẢNG CÁCH.....	13
BÀI 17. BỔ SUNG CHO ĐỦ CÁC SỐ TRONG KHOẢNG	14
BÀI 18. SỐ SMITH	14

BÀI 19. BỐN ĐIỂM TRÊN MẶT PHẪNG	15
BÀI 20. LẤY BỐT QUÂN CỜ	16
Sinh kế tiếp , đệ quy, quay lui , chia và trị	17
BÀI 1. XÂU NHỊ PHÂN KẾ TIẾP	17
BÀI 2. TẬP CON KẾ TIẾP.....	17
BÀI 3. HOÁN VỊ KẾ TIẾP.....	18
BÀI 4. XÂU NHỊ PHÂN TRƯỚC	19
BÀI 5. TẬP CON LIÊN KẾ PHÍA TRƯỚC	20
BÀI 6. LIỆT KÊ TỔ HỢP - SINH TỔ HỢP	20
BÀI 7. PHÂN TÍCH SỐ	22
BÀI 8. HOÁN VỊ XÂU KÝ TỰ	22
BÀI 9. DI CHUYỂN TRONG MÊ CUNG	23
BÀI 10. TỔ HỢP SỐ CÓ TỔNG BẰNG X.....	25
BÀI 11. DI CHUYỂN TRONG MA TRẬN	26
BÀI 12. SỐ NGUYÊN TỐ	26
BÀI 13. SẮP XẾP QUÂN HẬU.....	27
BÀI 14. TẬP HỢP.....	28
BÀI 15. MÁY ATM.....	29
BÀI 16. LŨY THỪA.....	29
BÀI 17. GẤP ĐÔI DÃY SỐ.....	30
BÀI 18. DÃY XÂU FIBONACI	31

BÀI 19. SỐ FIBONACCI THỨ N31

BÀI 20. DÃY CON LIÊN TIẾP CÓ TỔNG LỚN NHẤT32

 tham lam , chia và trị.....32

BÀI 1. LŨY THỪA MA TRẬN32

BÀI 2. LŨY THỪA ĐẢO.....33

BÀI 3. TÍCH HAI SỐ NHỊ PHÂN.....34

BÀI 4. SẮP XẾP CÔNG VIỆC.....35

BÀI 5. NỔI DÂY35

BÀI 6. SẮP ĐẶT XÂU KÝ TỰ.....36

BÀI 7. GIÁ TRỊ NHỎ NHẤT CỦA XÂU37

BÀI 8. SỐ MAY MẮN38

BÀI 9. SỐ NHỎ NHẤT CÓ N ƯỚC SỐ38

BÀI 10. KÝ TỰ ĐẶC BIỆT39

BÀI 11. NGƯỜI DU LỊCH.....40

BÀI 12. HỢP VÀ GIAO CỦA HAI MẢNG SỐ40

BÀI 13. TỔNG CẬP SỐ NGUYÊN TỐ.41

BÀI 14. MERGE SORT.....43

BÀI 15. BỔ SUNG PHẦN TỬ'43

Lập Trình cơ bản

BÀI 1. ƯỚC SỐ CHUNG LỚN NHẤT VÀ BỘI SỐ CHUNG NHỎ NHẤT

Ôn lại

Nội dung bài tập

Viết chương trình tìm ước số chung lớn nhất và bội số chung nhỏ nhất của hai số nguyên dương a,b.

Dữ liệu vào: Dòng đầu ghi số bộ test. Mỗi bộ test ghi trên một dòng 2 số nguyên a và b không quá 9 chữ số.

Kết quả: Mỗi bộ test ghi trên 1 dòng, lần lượt là USCLN, sau đó đến BSCNN.

Ví dụ:

Input	Output
2	
12 34	2 204
1234 5678	2 3503326

Note : 2 số nguyên a và b không quá 9 chữ số. => dung long long

Giải :

```
#include<iostream>
```

```
using namespace std;
```

```
void ucbc(long long a,long long b){
```

```
    long long m=a*b;
```

```
    while (a!=b){
```

```
        if(a>b) a=a-b;
```

```
        else b=b-a;
```

```
    }
```

```
    cout<<a <<" "<< m/a <<"\n";
```

```
}
```

```
int main(){
```

```
    long long n,a[1000],b[0100],count=0;
```

```
    cin>>n;
```

```
    for(int i=0;i<n;i++){
```

```
        cin>> a[count]>>b[count];
```

```
        count++;
```

```
    }
```

```

for(int i=0;i<count;i++){
    ucba(a[i],b[i]);
}

return 0;
}

```

BÀI 2. BẮT ĐẦU VÀ KẾT THÚC

Nội dung bài tập

Viết chương trình kiểm tra một số nguyên dương bất kỳ (2 chữ số trở lên, không quá 9 chữ số) có chữ số bắt đầu và kết thúc bằng nhau hay không.

Dữ liệu vào: Dòng đầu tiên ghi số bộ test. Mỗi bộ test viết trên một dòng số nguyên dương tương ứng cần kiểm tra.

Kết quả: Mỗi bộ test viết ra YES hoặc NO, tương ứng với bộ dữ liệu vào

Ví dụ:

Input	Output
2	
12451	YES
1000012	NO

Giải

```

#include<iostream>
using namespace std;
void huyinitwsolve(long long n){
    long long a[100],count=0;
    while(n>0){
        a[count++]=n%10;
        n/=10;
    }
    if (a[0]==a[count-1]) cout<<"YES\n";
    else cout<<"NO\n";
}

int main(){
    long long n,a[100],count=0;
    cin>>n;
    for(int i=0;i<n;i++){
        cin>> a[i];
        huyinitwsolve(a[i]);
    }
}

```

```

    }

    return 0;
}

```

BÀI 3. PHÉP CỘNG

Nội dung bài tập

Cho một phép toán có dạng $a + b = c$ với a,b,c chỉ là các số nguyên dương có một chữ số. Hãy kiểm tra xem phép toán đó có đúng hay không.

Dữ liệu vào: Dòng đầu ghi số bộ test. Mỗi test chỉ có một dòng ghi ra phép toán (gồm đúng 9 ký tự)

Kết quả: Ghi ra YES nếu phép toán đó đúng. Ghi ra NO nếu sai.

Ví dụ:

Input	Output
2	
1 + 2 = 3	YES
2 + 2 = 5	NO

Note:1. getline(cin,s); dùng getline(std::cin,s);cũng được

2. chuyển từ số ký tự sang số nguyên : int(s[0])-48 . ACII quy định số 0 là - 48

3. cin.ignore();vd, cin.ignore(256, '\n');, bỏ qua 256 ký tự đầu tiên hoặc tất cả các ký tự cho đến khi nó gặp phải phân định (ở đây\n),

Giải:

```

#include<iostream>
#include<string>
using namespace std;
void huyinitwsolve(){
    string s;
    getline(cin,s);
    if(int(s[0])+int(s[4])-48==int(s[8])) cout<<"YES\n";
    else cout<<"NO\n";
}

int main(){
    long long n;
}

```

```
cin>>n;
cin.ignore();
for(int i=0;i<n;i++){
huyinitwsolve();
}

return 0;
}
```

BÀI 4. SỐ TĂNG GIẢM

Nội dung bài tập

Một số được gọi là số tăng giảm nếu số đó có các chữ số thỏa mãn hoặc không giảm, hoặc không tăng từ trái qua phải. Hãy kiểm tra xem một số có phải số tăng giảm hay không.

Dữ liệu vào: Dòng đầu tiên ghi số bộ test. Mỗi bộ test viết trên một dòng một số nguyên dương cần kiểm tra, không quá 500 chữ số.

Kết quả: Mỗi bộ test viết ra chữ YES nếu đó đúng là số tăng giảm, chữ NO nếu ngược lại.

Input	Output
3	YES
23455667777777777778888888889999999	YES
9877777777777777777777776554422222111111111000	NO
43435312432543657657658769898097876465465687987	

```
Giải:
#include<iostream>
#include<string>
using namespace std;
int huyinitwsolve(){
    string s;
    cin>>s;
    for(int i=0;i<s.length()-2;i++){
        if( int(s[i]< int(s[i+1])&&int(s[i+1])>int(s[i+2])) return 0;
        if( int(s[i]> int(s[i+1])&&int(s[i+1])<int(s[i+2])) return 0;
    }
    return 1;
}
int main(){
```

```
long long n;
cin>>n;
cin.ignore();
for(int i=0;i<n;i++){
    if(huyinitwsolve()==1) cout<<"YES\n";
    else cout<<"NO\n";
}

return 0;
}
```

BÀI 5. LIỆT KÊ VÀ ĐẾM

Nội dung bài tập

Cho một dãy các số nguyên dương không quá 9 chữ số, mỗi số cách nhau một khoảng trống. Hãy tìm các số chỉ bao gồm các *chữ số nguyên tố* và đếm số lần xuất hiện của các số đó.

Input: Gồm không quá 50000 số nguyên dương, các số đều không quá 9 chữ số.

Output: Ghi ra các số chỉ có các chữ số nguyên tố kèm theo số lần xuất hiện. Các số được liệt kê theo thứ tự xuất hiện (tức là số nào xuất hiện trước in ra trước).

Ví dụ:

Input	Output
123 321 23456 123 123 23456 3523 123 321 4567 8988	3523 4
878 7654 9899 3456 123 678 999 77 3456 878	77 2
987654321 4546 63543 4656 13432 4563 123471	57275 1
659837 57275 34355 878 9087 77 98534 3456 23132	
3523 3523 3523	

```
Giải :
#include<bits/stdc++.h>
using namespace std;
int kt(int n){
    while(n>0){
        int t=n%10;
        if(t!=2&& t!=3&& t!=5 && t!=7) return 0;
```

```

        n/=10;
    }
    return 1;
}
int tim(int x,int a[50005],int n){
    for(int i=0;i<n;i++){
        if(a[i]==x) return i;
    }
    return -1;
}
int main(){
    int x,a[50005],b[50005],i,n=0;
    while(cin>>x){
        if(kt(x)==1){
            i=tim(x,a,n);
            if(i!=-1){
                a[n]=x;
                b[n]=1;
                n++;
            }
            else b[i]++;
        }
        for(int i=0;i<n;i++) cout<<a[i]<<" "<<b[i]<<endl;
        return 0;
    }
}

```

BÀI 6. SỐ ĐẦU TIÊN BỊ LẬP

Nội dung bài tập

Cho dãy số $A[]$ gồm có N phần tử. Nhiệm vụ của bạn là hãy tìm số xuất hiện nhiều hơn 1 lần trong dãy số và số thứ tự là nhỏ nhất.

Input: Dòng đầu tiên là số lượng bộ test T ($T \leq 10$). Mỗi test gồm số nguyên N ($1 \leq N \leq 100\,000$), số lượng phần tử trong dãy số ban đầu. Dòng tiếp theo gồm N số nguyên $A[i]$ ($0 \leq A[i] \leq 10^9$).

Output: Với mỗi test in ra đáp án của bài toán trên một dòng. Nếu không tìm được đáp án, in ra “NO”.

Ví dụ:

Input	Output
2 7 10 5 3 4 3 5 6 4 1 2 3 4	5 NO

Giải thích test 1: Cả 5 và 3 đều xuất hiện 2 lần, nhưng số 5 có số thứ tự nhỏ hơn.

Giải :

```

#include<iostream>
#include<string>
using namespace std;
int huyinitwsolve(){
    int n;
    cin>>n;
    long long a[n];
    for(int i=0;i<n;i++){
        cin>>a[i];
    }
    int count=0;
    for(int i=0;i<n;i++){
        if(i==n-1)cout<<"NO\n";
        count=0;
        for(int j=i+1;j<n;j++){
            if(a[j]==a[i]){
                count++;
            }
        }
        if(count>0) {
            cout<<a[i]<<"\n";break;
        }
    }
}
int main(){

```

```

        int n;
        cin>>n;
        for(int i=0;i<n;i++){
            huyinitwsolve();
        }
        return 0;
    }

Cách 2:
#include<iostream>
#include<string>
using namespace std;
int huyinitwsolve(){
    int n;
    cin>>n;
    long long a[n];
    for(int i=0;i<n;i++){
        cin>>a[i];
    }
    int temp=0;
    for(int i=0;i<n;i++){
        if(i==n-1)cout<<"NO\n";
        for(int j=i+1;j<n;j++){
            if(a[j]==a[i]){
                cout<<a[i]<<"\n";
                temp=1;
                break;
            }
        }
        if(temp==1) break;
    }
}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
}

```

```

    }
    return 0;
}

```

BÀI 7. GHÉP HÌNH

Ôn lại

Nội dung bài tập

Cho ba hình chữ nhật. Các bạn được phép xoay hình nhưng không được phép xếp chồng lẫn lên nhau, hỏi 3 hình chữ nhật đó có thể ghép thành một hình vuông được hay không

Input: Dòng đầu ghi số bộ test. Mỗi test có ba dòng, mỗi dòng ghi hai số nguyên dương là chiều rộng và chiều cao của hình chữ nhật (các số đều không quá 100).

Output: Ghi ra YES nếu có thể tạo thành hình vuông, NO nếu không thể.

Ví dụ:

Input	Output
1	YES
8 2	
1 6	
7 6	

Note : #include <bits/stdc++.h> gọi tắt cả các thư viện c++ và c

- <https://tichpx.wordpress.com/2015/09/23/10729-ghiep-hinh/?fbclid=IwAR07Qeq9GrTIEjhd6KqBf5bbeOJv9zsvqoGFt0PDhzZl1kpi5FFCATgDSAQ>

Giải :

```

#include <bits/stdc++.h>
using namespace std;
int kt(int dientich){
    int canh=sqrt(dientich);
    if(canh*canh==dientich) return canh;
    else return 0;
}

void huyinitwsolve(){
    int a,b,c,d,e,f;
    cin>>a>>b>>c>>d>>e>>f;
    int x=a*b+c*d+e*f;
    int y=kt(x);
}

```

```

if(y==0) cout<<"NO\n";
else{
    if(a>b) swap(a,b);
    if(c>d) swap(c,d);
    if(e>f) swap(e,f);
    if(d==y){ swap(a,c);swap(b,d);}
    if(f==y){ swap(a,e);swap(b,f);}
    if(b==y){
        a=y-a;
        if( (c==a&&e==a) ||(c==a&&f==a) ||(d==a&&e==a)
||(d==a&&f==a) ) cout<<"YES\n";
        else cout <<"NO\n";
    } else cout<<"NO\n";
}

}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
}

```

BÀI 8. CHIA HẾT CHO 2

Ôn lại

Nội dung bài tập

Cho số nguyên dương N .

Nhiệm vụ của bạn là hãy xác định xem có bao nhiêu ước số của N chia hết cho 2.

Input: Dòng đầu tiên là số lượng bộ test T ($T \leq 100$). Mỗi bộ test gồm một số nguyên N ($1 \leq N \leq 10^9$)

Output: Với mỗi test, in ra đáp án tìm được trên một dòng.

Ví dụ:

Input:	Output:
2	0
9	3
8	

Giải:

```

#include <bits/stdc++.h>
using namespace std;

```

```

int huyinitwsolve(long long n){
    int count=1;
    if(n%2==1) return 0;
    else {
        for(int i=2;i<=sqrt(n);i++){
            if(n%i==0){
                if(i%2==0)count++;
                int j=n/i;
                if(j%2==0&&j!=i)count++;
            }
        }
        return count;
    }
}

```

```

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        long long n;
        cin>>n;
        cout<<huyinitwsolve(n)<<endl;
    }
}

```

BÀI 9. ĐỔI TIỀN

Tại ngân hàng có các mệnh giá bằng 1, 2, 5, 10, 20, 50, 100, 200, 500, 1000. Tổng số tiền cần đổi có giá trị bằng N. Hãy xác định xem có ít nhất bao nhiêu tờ tiền sau khi đổi tiền?

Input:
Dòng đầu tiên là số lượng bộ test T (T ≤ 50). Mỗi test gồm 1 số nguyên N (1 ≤ N ≤ 100 000).

Output: Với mỗi test, in ra đáp án trên một dòng.

Ví dụ:

Input:	Output
2	
70	2
121	3

```
Giải:
#include<bits/stdc++.h>
using namespace std;

void huyinitwsolve(){
    int n,count=0;
    cin>>n;
    int a[10]={ 1000,500,200,100,50,20,10,5,2,1 };
    for(int i=0;i<10;i++){
        if(n/a[i]!=0){
            count++;
            n-=a[i];
            i--;
        }
    }
    cout<<count<<endl;
}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
```

```
huyinitwsolve();
}
}
```

BÀI 10. NHÀM CHỮ SỐ

Nội dung bài tập
Trong một buổi học toán, giáo viên viết 2 số nguyên, A và B, và yêu cầu Tèo thực hiện phép cộng. Tèo không bao giờ tính toán sai, nhưng thỉnh thoảng cậu ta chép các con số một cách không chính xác. Lỗi duy nhất của là ghi nhầm '5' thành '6' hoặc ngược lại. Cho hai số, A và B, tính tổng nhỏ nhất và lớn nhất mà Tèo có thể nhận được.

Input: Có một dòng chứa hai số nguyên dương A và B (1 ≤ A, B ≤ 1 000 000).

Output: In ra 2 số nguyên cách nhau một dấu cách, tổng nhỏ nhất và lớn nhất có thể nhận được.

Ví dụ:

Test 1	Test 2	Test 3
Input: 11 25	Input: 1430 4862	Input: 16796 58786
Ouput: 36 37	Ouput: 6282 6292	Ouput: 74580 85582

```
Chú ý : HÀM chuyển từ một chuỗi sang một số : int convert(string s){
    int res=0;
    for(int i=0;i<s.length();i++){
        res=10*res+s[i]-48;
    }
    return res;
}

Giải
#include<bits/stdc++.h>
#include<stdlib.h>
#include <iostream>
#include <string>
using namespace std;
```



```

int convert(string s){
    int res=0;
    for(int i=0;i<s.length();i++){
        res=10*res+s[i]-48;
    }
    return res;
}

int main(){
    string a,b;
    cin>>a>>b;
    int min=0,max=0;
    for(int i=0;i<a.length();i++){
        if(a[i]=='6') a[i]='5';
    }
    for(int i=0;i<b.length();i++){
        if(b[i]=='6') b[i]='5';
    }
    cout<<convert(a)+convert(b) <<" ";
    for(int i=0;i<a.length();i++){
        if(a[i]=='5') a[i]='6';
    }
    for(int i=0;i<b.length();i++){
        if(b[i]=='5') b[i]='6';
    }
    cout<<convert(a)+convert(b) <<endl;
}

```

Cách 2:

```

#include <bits/stdc++.h>
using namespace std;

```

```

int main(){
    string a,b;
    cin>>a>>b;

    int minA=0, minB=0, maxA=0, maxB=0;

```

```

for(int i=0 ; i<a.length() ; i++){
    if(a[i] == '5'){
        maxA = maxA *10 + a[i] - 48 + 1;
        minA = minA *10 + a[i] - 48;
    }
    else if(a[i] == '6'){
        maxA = maxA * 10 + a[i] - 48;
        minA = minA * 10 + a[i] - 48 - 1;
    }
    else {
        maxA = maxA * 10 + a[i] - 48;
        minA = minA * 10 + a[i] - 48;
    }
}

```

```

for(int i=0 ; i<b.length() ; i++){
    if(b[i] == '5'){
        maxB = maxB *10 + b[i] - 48 + 1;
        minB = minB *10 + b[i] - 48;
    }
    else if(b[i] == '6'){
        maxB = maxB * 10 + b[i] - 48;
        minB = minB * 10 + b[i] - 48 - 1;
    }
    else {
        maxB = maxB * 10 + b[i] - 48;
        minB = minB * 10 + b[i] - 48;
    }
}

```

```

cout<<minA + minB << " "<<maxA + maxB;

```

```

return 0;

```

```

}

```

BÀI 11. TỔNG NHỎ NHẤT

Nội dung bài tập

Cho mảng $A[]$ gồm các số từ 0 đến 9. Nhiệm vụ của bạn là tìm tổng nhỏ nhất của hai số được tạo bởi các số trong mảng $A[]$. Chú ý, tất cả các số trong mảng $A[]$ đều được sử dụng để tạo nên hai số.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo đưa vào N số $A[i]$ tương ứng với các phần tử của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 20$; $0 \leq A[i] \leq 9$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
6	
6 8 4 5 2 3	604
5	82
5 3 0 7 4	

Note :

- Hàm `sort` : `sort(a,a+n)`; của **Thư viện algorithm** toán tử `<=` đã định nghĩa.

Giải :

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void huyinitwsolve(){
    int n;
    long long suma=0,sumb=0;
    cin>>n;
    int a[n];
    for(int i=0;i<n;i++) cin>>a[i];
```

```
    sort(a,a+n);
    for(int i=0;i<n;i++){
        if( i%2==0) suma=10*suma+a[i];
        else sumb=10*sumb+a[i];
    }
    cout<<suma+sumb<<endl;
}
```

```
int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
}
```

BÀI 12. SỐ KHỐI LẬP PHƯƠNG

Nội dung bài tập

Một số X được gọi là số khối lập phương nếu X là lũy thừa bậc 3 của số Y ($X = Y^3$). Cho số nguyên dương N , nhiệm vụ của bạn là tìm số khối lập phương lớn nhất bằng cách loại bỏ đi các chữ số của N . Ví dụ số 4125 ta có kết quả là $125 = 5^3$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Nếu không tìm được đáp án in ra -1.

Ví dụ:

Input	Output
2	
4125	125
976	-1

Giải :

```
#include<iostream>
```

```

#include<string.h>
#include<math.h>
using namespace std;
int ok(long long n,char s[20]){
    char s1[20];
    int j,i=0,j1,dem=0,x=0;
    while(n>0){
        s1[i]=(char)(48+n%10);
        n=n/10;
        i++;
    }
    for(j=i-1;j>=0;j--){
        for(j1=x;j1<strlen(s);j1++){
            if(s1[j]==s[j1]){
                dem++;
                x=j1+1;
                break;
            }
        }
    }
    if(dem==i)return 1;
    else return 0;
}
void inra(char s[20]){
    long long max=-1,i,x=0;
    for(i=1;i*i*i<=pow(10,strlen(s));i++){
        if(ok(i*i*i,s)==1) max=i*i*i;
    }
    cout<<max<<endl;
}
using namespace std;
int main(){
    int n;char s[20];
    cin>>n;
    while(n--){
        cin>>s;

```

```

        inra(s);
    }
}

```

BÀI 13. SỐ NHỎ NHẤT

Nội dung bài tập

Cho hai số nguyên dương S và D, trong đó S là tổng các chữ số và D là số các chữ số của một số. Nhiệm vụ của bạn là tìm số nhỏ nhất thỏa mãn S và D? Ví dụ với S = 9, D = 2 ta có số nhỏ nhất thỏa mãn S và D là 18.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ 2 số S và D được viết trên một dòng.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq S, D \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Nếu không có đáp án, in ra -1.

Ví dụ:

Input	Output
2	18
9 2	299
20 3	

Ý tưởng: - nếu tổng > 9 * số chữ số thì in ra -1;

- Tiếp tục vòng lặp số 9 ở cuối cho đến khi Nếu 9 < tổng thì : cho chữ số cuối là (tổng - 1) ; các chữ số ở giữa là 0; và chữ số đầu là 1

Giải:

```

#include<bits/stdc++.h>
using namespace std;

```

```

int huyinitwsolve(int s,int d){

```

```

    string re[d];
    for(int i=d-1;i>=0;i--){
        if(s>9){
            re[i]='9';
            s-=9;
        }
    }
}

```

```

    else{
        if(i!=0){
            re[i]=s-1+48;
            s=1;
        }
        else re[i]=s+48 ;
    }
}
for(int i=0;i<d;i++) cout<<re[i];cout<<endl;
}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        int s,d;
        cin>>s>>d;
        if(9*d<s) {
            cout<<-1<<endl;
        }
        else huyinitwsolve(s,d);
    }
}

```

BÀI 14. PHÂN SỐ ĐƠN VỊ

Nội dung bài tập

Một phân số đơn vị nếu tử số của phân số đó là 1. Mọi phân số nguyên dương đều có thể biểu diễn thành tổng các phân số đơn vị. Ví dụ $\frac{2}{3} = \frac{1}{2} + \frac{1}{6}$. Cho phân số nguyên dương P/Q bất kỳ ($P < Q$), hãy biểu diễn phân số nguyên dương thành tổng phân số đơn vị.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ đôi tử số P và mẫu số Q của phân số nguyên dương được viết trên một dòng.
- T, P, Q thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq P, Q \leq 100$.

Output:

- Đưa ra đáp án tìm được trên 1 dòng, theo dạng “ $\frac{1}{a} + \frac{1}{b} + \dots$ ”

Ví dụ:

Input	Output
2	
2 3	$\frac{1}{2} + \frac{1}{6}$
1 3	$\frac{1}{3}$

-Ý tưởng: $\frac{2}{3}$ chính là tử/mẫu= $\frac{1}{(3/2)}=\frac{1}{\text{res} + x}$

Trong đó res chạy từ 2 $\nleftrightarrow$ mau/tu+1

Từ đó suy ra số X với tử số tu=res*tu-mau; và mẫu số :mau=res*mau;

Giải :

```

#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    long long tu,mau;
    cin>>tu>>mau;
    long long res=0;
    while(1){
        if(mau%tu==0){
            cout<<"1/"<<(mau/tu)<<endl;
            break;
        }
        else {
            res=mau/tu+1;
            cout<<"1/"<<res<<" + ";
            tu=res*tu-mau;
            mau=res*mau;
        }
    }
}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
    return 0;
}

```

}
BÀI 15. SẮP XẾP XEN KÊ

Nội dung bài tập

Cho mảng A[] gồm n số nguyên khác nhau. Hãy đưa ra các phần tử của mảng theo khuôn dạng lớn nhất, nhỏ nhất, lớn thứ hai, nhỏ thứ 2, ... Ví dụ với A[] = {9, 7, 12, 8, 6, 5} ta đưa ra : 12, 5, 9, 6, 8, 7.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n; dòng tiếp theo là n số A [i] của mảng A []; các số được viết cách nhau một vài khoảng trống.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
7	
7 1 2 3 4 5 6	7 1 6 2 5 3 4
8	9 1 8 2 7 3 6 4
1 6 9 4 3 7 8 2	

Giải :

```
#include<bits/stdc++.h>
using namespace std;
```

```
int huyinitwsolve(){
    int n;
    cin>>n;
    int a[n];
    for(int i=0;i<n;i++) cin>>a[i];
    sort(a,a+n);
    if(n%2==0) for(int i=0;i<n/2;i++)cout<<a[n-i-1]<<" "<<a[i]<<" ";
    else {
        for(int i=0;i<n/2;i++)cout<<a[n-i-1]<<" "<<a[i]<<" ";
        cout<<a[n/2]<<endl;
    }
}
```

```
int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
}
```

BÀI 16. SẮP XẾP THEO KHOẢNG CÁCH

Nội dung bài tập

Cho mảng A[] gồm n phần tử và số X. Hãy đưa sắp xếp các phần tử của mảng theo trị tuyệt đối của $|X - A[i]|$. Ví dụ với A[] = {10, 5, 3, 9, 2} và X = 7 ta đưa ra mảng được sắp xếp theo nguyên tắc kê trên: A[] = {5, 9, 10, 3, 2} vì $|7-10|=3$, $|7-5|=2$, $|7-3|=4$, $|7-9|=2$, $|7-2|=5$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n và X; dòng tiếp theo là n số A [i] của mảng A []; các số được viết cách nhau một vài khoảng trống.
- T, n, X thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, X, A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
5 7	
10 5 3 9 2	5 9 10 3 2
5 6	5 4 3 2 1
1 2 3 4 5	

Note : bài này đưa số cuối lớn nhất lên trước =>I chạy đến n, j chạy đến n-1;

Giải :

```
#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    int n,x;
```

```

cin>>n>>x;
int a[n];
for(int i=0;i<n;i++) cin>>a[i];
for(int i=0;i<n;i++){
    for(int j=0;j<n-1;j++){
        if(abs(x-a[j])>abs(x-a[j+1])) swap(a[j],a[j+1]);
    }
}
for(int i=0;i<n;i++) cout<<a[i]<<" "; cout<<endl;
}

int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
    return 0;
}

```

BÀI 17. BỔ SUNG CHO ĐỦ CÁC SỐ TRONG KHOẢNG

Nội dung bài tập

Cho mảng $A[]$ gồm n số nguyên dương. Gọi L, R là max và min các phần tử của $A[]$. Nhiệm vụ của bạn là tìm số phần tử cần thiết cần thêm vào mảng để mảng có đầy đủ các số trong khoảng $[L, R]$. Ví dụ $A[] = \{5, 7, 9, 3, 6, 2\}$ ta nhận được kết quả là 2 tương ứng với các số còn thiếu là 4, 8.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n , tương ứng với số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq n, A[i] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
--------	---------

2	
5	1
4 5 3 8 6	0
3	
2 1 3	

Giải :

```

#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    int n,count=0;
    cin>>n;
    int a[n];
    for(int i=0;i<n;i++) cin>>a[i];
    sort(a,a+n);
    for(int i=0;i<n-1;i++) if (a[i+1]!=a[i]) count++;
    cout<<a[n-1]-a[0]-count<<endl;
}
int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
    return 0;
}

```

BÀI 18. SỐ SMITH

Nội dung bài tập

Một số Smith là một số tự nhiên thỏa mãn tổng các chữ số của nó bằng với tổng các chữ số của các nhân tử. Một vài số Smith đó là 4, 22, 27, 58, 85, 94, ...

Nhiệm vụ của bạn là hãy xác định xem số nguyên N đã cho có là số Smith hay không?

Ví dụ 1:

$$378 = 2 \times 3 \times 3 \times 3 \times 7$$

Tổng các chữ số là $3 + 7 + 8 = 18$.

Tổng các chữ số của các nhân tử là $2 + 3 + 3 + 3 + 7 = 18$.

Ví dụ 2:

4937775 = 3 x 5 x 5 x 5 x 65837.

Ta có $4+9+3+7+7+7+5 = 42 = 3+5+5+6+5+8+3+7$.

Input

- Dòng đầu ghi số bộ test.
- Mỗi test là một số nguyên dương N ($1 < N < 2^{32}$).

Output

- In ra “YES” nếu số đã cho là số Smith, in ra “NO” trong trường hợp ngược lại.

Ví dụ:

Input	Output
2	YES
378	NO
120	

Giới :

```
#include<bits/stdc++.h>
using namespace std;
int sum(int n){
    int temp=0;
    while(n>0){
        temp +=n%10;
        n/=10;
    }
    return temp;
}
void huyinitwsolve(){
    int n,count=0;
    cin>>n;
    int sum1=0,sum2=0;
    sum1=sum(n);
    // cout<<sum1<<endl;
    int i=2;
    while(n>1){
        if(n%i==0){
            sum2+=sum(i);
```

```
        n/=i;
    }
    else i++;
}
if(sum1==sum2) cout<<"YES"<<endl;
else cout<<"NO"<<endl;
}
int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
    return 0;
}
```

BÀI 19. BÓN ĐIỂM TRÊN MẶT PHẪNG

Nội dung bài tập

Cho 4 điểm trong không gian 3 chiều. Nhiệm vụ của bạn là kiểm tra xem chúng có cùng nằm trên một mặt phẳng hay không? Nếu có in ra “YES”, in ra “NO” trong trường hợp ngược lại.

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 10000$).

Mỗi test gồm 4 dòng, lần lượt là tọa độ nguyên $x[i]$, $y[i]$, $z[i]$ của các điểm. ($-1000 \leq x[i], y[i], z[i] \leq 1000$).

Output:

Với mỗi test, in ra đáp án tìm được trên một dòng.

Input:	Output
3	
1 2 0	
2 3 0	
4 0 0	YES
0 0 0	YES
1 1 1	NO
2 2 2	
3 3 3	
4 4 4	

5 6 7 -8 -9 -10 12 19 0 3 1 5	
--	--

```

Giải :
#include<bits/stdc++.h>
using namespace std;
int huyinitwsolve(){
    int a[5][4];
    for(int i=0;i<4;i++)    for(int j=0;j<3;j++) cin>>a[i][j];

    for(int i=1;i<4;i++){
        for(int j=0;j<3;j++){
            a[i][j]=a[i][j]-a[0][j];
        }
    }

    a[0][0]=a[1 ][1]*a[2 ][2 ]-a[1 ][2 ]*a[2 ][1 ];
    a[0][1]=a[1 ][2 ]*a[2 ][0 ]-a[1 ][0 ]*a[2 ][2 ];
    a[0][2]=a[1 ][0 ]*a[2 ][1 ]-a[1 ][1 ]*a[2 ][0 ];

    if(    a[0][0]*a[3][0]+a[0][1]*a[3][1]+a[0][2]*a[3][2]==0)
cout<<"YES\n";
    else cout<<"NO\n";

}

int main(){
    int n ;
    cin>>n;
    for(int i=0;i<n;i++) huyinitwsolve();
    return 0;
}

```

BÀI 20. LẤY BÓT QUÂN CỜ
 Nội dung bài tập

Trên bàn có C quân cờ. Có hai đối thủ chơi lần lượt. Mỗi lượt, người chơi sẽ lấy khỏi bàn từ 1 đến M quân cờ. Người thắng cuộc là người lấy được quân cờ cuối cùng.

Biết rằng hai người chơi đều chơi tối ưu. Hãy xác định xem ai là người thắng cuộc?

Input:
 Dòng đầu tiên là số lượng bộ test T (T <= 1000).
 Mỗi test gồm 2 số nguyên C và M (0 <= C <= 1000, 1 <= M <= 1000).
Output:
 In ra “First” nếu người đi trước là người chiến thắng, in ra “Second” nếu người chơi sau là người chiến thắng.

Test ví dụ:

Input:	Output
4	
20 9	Second
1 5	First
7 5	First
0 3	Second

```

Giải:
#include<bits/stdc++.h>
using namespace std;
int huyinitwsolve(){
    int c,m;
    cin>>c>>m;
    if(c%(m+1)==0) cout<<"Second\n";
    else cout<<"First\n";

}

int main(){
    int n ;
    cin>>n;
    for(int i=0;i<n;i++) huyinitwsolve();
    return 0;
}

```


Sinh kế tiếp , đệ quy, quay lui , chia và trị

BÀI 1. XÂU NHỊ PHÂN KẾ TIẾP

Nội dung bài tập

Cho xâu nhị phân $X[]$, nhiệm vụ của bạn là hãy đưa ra xâu nhị phân tiếp theo của $X[]$. Ví dụ $X[] = "010101"$ thì xâu nhị phân tiếp theo của $X[]$ là "010110".

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu nhị phân X .
- $T, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(X) \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	
010101	010110
111111	000000

```
#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    string s;
    cin>>s;
    int n=s.length();
    while(n--){
        if(s[n]=='0'){
            s[n]='1';
            break;
        }
        else s[n]='0';
    }

    cout<<s<<endl;
}
int main(){
    int n;
    cin>>n;
```

```
while(n--){
    huyinitwsolve();
}
return 0;
```

BÀI 2. TẬP CON KẾ TIẾP

Nội dung bài tập

Cho hai số N, K và một tập con K phần tử $X[] = (X_1, X_2, \dots, X_K)$ của $1, 2, \dots, N$. Nhiệm vụ của bạn là hãy đưa ra tập con K phần tử tiếp theo của $X[]$. Ví dụ $N=5, K=3, X[] = \{2, 3, 4\}$ thì tập con tiếp theo của $X[]$ là $\{2, 3, 5\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là hai số N và K ; dòng tiếp theo đưa vào K phần tử của $X[]$ là một tập con K phần tử của $1, 2, \dots, N$.
- $T, K, N, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq K \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	
5 3	2 3 4
1 4 5	1 2 3
5 3	
3 4 5	

Cách 1:

```
#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    int n,k;
    cin>>n>>k;
    int a[k];
    for(int i=1;i<=k;i++)cin>>a[i];
    int x=k,check=0;
    while(x>0){
        if(a[x]!=n-k+x){
```

```

        a[x]++;
        for(int i=x+1;i<=k;i++){
            a[i]=a[i-1]+1;
        }
        check=1;
        break;
    }
    x--;
}
if(check==1) for(int i=1;i<=k;i++) cout<<a[i]<<" ";
else for(int i=1;i<=k;i++) cout<<i<<" ";
cout<<endl;
}
int main(){
    int n;
    cin>>n;
    while(n--){
        huyinitwsolve();
    }
    return 0;
}

```

CÁCH 2:

```

#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    int n,k;
    cin>>n>>k;
    int a[1000];
    for(int i=1;i<=k;i++)cin>>a[i];
    int x=k,check=0;
    for(int i=k;i>=1;i--){
        if(a[i]!=n-k+i){
            a[i]++;
            for(int j=i+1;j<=k;j++){
                a[j]=a[j-1]+1;
            }
        }
    }
}

```

```

        check=1;
        break;
    }
}
if(check==1) for(int i=1;i<=k;i++) cout<<a[i]<<" ";
else for(int i=1;i<=k;i++) cout<<i<<" ";
cout<<endl;
}
int main(){
    int n;
    cin>>n;
    while(n--){
        huyinitwsolve();
    }
    return 0;
}

```

BÀI 3. HOÁN VỊ KẾ TIẾP

Nội dung bài tập

Cho số tự nhiên N và một hoán vị $X[]$ của $1, 2, \dots, N$. Nhiệm vụ của bạn là đưa ra hoán vị tiếp theo của $X[]$. Ví dụ $N=5$, $X[] = \{1, 2, 3, 4, 5\}$ thì hoán vị tiếp theo của $X[]$ là $\{1, 2, 3, 5, 4\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số N ; dòng tiếp theo đưa vào hoán vị $X[]$ của $1, 2, \dots, N$.
- $T, N, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	
5	
1 2 3 4 5	1 2 3 5 4
5	1 2 3 4 5
5 4 3 2 1	

```
#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    int n;
    cin>>n;
    int a[1000];
    for(int i=1;i<=n;i++) cin>>a[i];
    int check=0,mark=0;

    for(int i=n;i>1;i--){
        if(a[i-1]<a[i]) {
            mark=i-1;
            check=1;
            break;
        }
    }
    // cout<<check<<endl;
    for(int i=n;i>mark;i--){
        if(a[i]>a[mark]) {
            swap(a[i],a[mark]);break;
        }
    }

    for(int i=mark+1;i<n;i++){
        for(int j=mark+2;j<=n;j++){
            if(a[j]>a[i]) swap(a[i],a[j]);
        }
    }

    if(check==1)for(int i=1;i<=n;i++) cout<<a[i]<<" ";
    else for(int i=1;i<=n;i++) cout<<i<<" "; cout<<endl;
}

int main(){
    int n;
```

```
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
    return 0;
}
```

BÀI 4. XÂU NHỊ PHÂN TRƯỚC

Nội dung bài tập

Cho xâu nhị phân $X[]$, nhiệm vụ của bạn là hãy đưa ra xâu nhị phân trước của $X[]$. Ví dụ $X[] = "111111"$ thì xâu nhị phân trước của $X[]$ là $"111110"$. Với xâu $X[] = "000001"$ thì xâu nhị phân trước của $X[]$ là $"000000"$. Chú ý: nếu xâu dữ liệu trong input là xâu đầu tiên thì trước nó sẽ là xâu cuối cùng.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu nhị phân X .
- $T, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(X) \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
010101	010100
111111	111110

```
#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    string s;
    cin>>s;
    int n=s.length();
    while(n--){
        if(s[n]=='1'){
            s[n]='0';
            break;
        }
        else s[n]='1';
    }
```

```

    }

    cout<<s<<endl;
}

int main(){
    int n;
    cin>>n;
    while(n--){
        huyinitwsolve();
    }
    return 0;
}

```

BÀI 5. TẬP CON LIÊN KÈ PHÍA TRƯỚC

Nội dung bài tập

Cho hai số N, K và một tập con K phần tử $X[] = (X_1, X_2, \dots, X_K)$ của $1, 2, \dots, N$. Nhiệm vụ của bạn là hãy đưa ra tập con K phần tử trước đó của $X[]$. Ví dụ $N=5, K=3, X[] = \{2, 3, 5\}$ thì tập con trước đó của $X[]$ là $\{2, 3, 4\}$. Chú ý nếu tập con trong input là đầu tiên thì trước đó là tập con cuối cùng.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là hai số N và K ; dòng tiếp theo đưa vào K phần tử của $X[]$ là một tập con K phần tử của $1, 2, \dots, N$.
- $T, K, N, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq K \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
5 3	
2 3 5	2 3 4
5 3	3 4 5
1 2 3	

```

#include<bits/stdc++.h>
#define ll long long

```

```

#define mod 1000000007
using namespace std;
void huyinitwsolve(){
    int n,k;
    int a[10005]={0};
    cin>>n>>k;
    for(int i=1;i<=k;i++){
        cin>>a[i];
    }
    int vt=k;
    while(a[vt]==a[vt-1]+1) vt--;
    // cout<<vt;
    if(vt==0){
        for(int i=k;i>=1;i--) cout<<n-i+1<<" ";
    }
    else {
        a[vt]--;
        for(int i=vt+1;i<=k;i++) a[i]=n-k+i;
        for(int i=1;i<=k;i++) cout<<a[i]<<" ";
    }
}

int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
        cout<<endl;
    }
    return 0;
}

```

BÀI 6. LIỆT KÊ TỔ HỢP - SINH TỔ HỢP

Nội dung bài tập

Cho hai số nguyên dương N và K . Nhiệm vụ của bạn là hãy liệt kê tất cả các tập con K phần tử của $1, 2, \dots, N$. Ví dụ với $N=5, K=3$ ta có 10 tập con của $1,$

2, 3, 4, 5 như sau: {1, 2, 3}, {1, 2, 4}, {1, 2, 5}, {1, 3, 4}, {1, 3, 5}, {1, 4, 5}, {2, 3, 4}, {2, 3, 5}, {2, 4, 5}, {3, 4, 5}.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một cặp số tự nhiên N, K được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq k \leq n \leq 15$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	123 124 134 234
4 3	123 124 125 134 135 145 234 235 245 345
5 3	

C1:

```
#include<bits/stdc++.h>
using namespace std;
void out(int a[1000],int n,int k){
    for(int i=1;i<=k;i++){
        cout<<a[i];
    }
    cout<<" ";
}
void huyinitwsolve(){
    int n,k;
    cin>>n>>k;
    int a[1000];
    for(int i=1;i<=k;i++) a[i]=i; out(a,n,k);

    for(int i=k;i>=1;i--){
        if(a[i]!=n-k+i){
            a[i]++;
            for(int j=i+1;j<=k;j++){
                a[j]=a[j-1]+1;
            }
        }
    }
}
```

```
out(a,n,k);
i=k+1;
```

```
}
}
cout<<endl;
```

```
}
int main(){
    int n;
    cin>>n;
    while(n--){
        huyinitwsolve();
    }
    return 0;
```

```
}
C2: quay lui
#include<bits/stdc++.h>
using namespace std;
int n,k, a[100]={ };
void init(){
    for(int i=1;i<=k;i++) a[i]=i;
}
void out(){
    for(int i=1;i<=k;i++) cout<<a[i];
    cout<<" ";
}
void Try(int i){
    for(int j=a[i-1]+1;j<=n-k+i;j++){
        a[i]=j;
        if(i==k) out();
        else Try(i+1);
    }
}
int main(){
```

```

int t;
cin>>t;
while(t--){
    cin>>n>>k;
    init();
    out();
    Try(1);
}
return 0;
}

```

BÀI 7. PHÂN TÍCH SỐ

Nội dung bài tập

Cho số nguyên dương N . Nhiệm vụ của bạn là hãy liệt kê tất cả các cách phân tích số tự nhiên N thành tổng các số tự nhiên nhỏ hơn hoặc bằng N . Phép hoán vị vừa một cách được xem là giống nhau. Ví dụ với $N = 5$ ta có kết quả là: (5), (4, 1), (3, 2), (3, 1, 1), (2, 2, 1), (2, 1, 1, 1), (1, 1, 1, 1, 1).
Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T, N \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	(4) (3 1) (2 2) (2 1 1) (1 1 1 1)
4	(5) (4 1) (3 2) (3 1 1) (2 2 1) (2 1 1 1) (1 1 1 1 1)
5	

Giải :

```

#include<bits/stdc++.h>
using namespace std;
int n, a[100]={};

```

```

void out(int n){
    cout<<"(";

```

```

        for(int i=1;i<n;i++) cout<<a[i]<<" ";
        cout<<a[n]<<"(");
    }

    void solve(int x,int sum,int k){
        for(int i=k;i>=1;i--){
            if(sum+i<=n){
                sum+=i;
                a[x]=i;
                if(sum==n) out(x);
            }
            else solve(x+1,sum,i);
            sum-=i;
        }
    }

    int main(){
        int t;
        cin>>t;
        while(t--){
            cin>>n;
            solve(1,0,n);
            cout<<endl;
        }
        return 0;
    }
}

```

BÀI 8. HOÁN VỊ XÂU KÝ TỰ

Nội dung bài tập

Cho chuỗi ký tự S bao gồm các ký tự in hoa khác nhau. Hãy đưa ra tất cả các hoán vị của chuỗi ký tự S . Ví dụ $S = "ABC"$ ta có kết quả {ABC ACB BAC BCA CAB CBA}.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test là một xâu ký tự S được viết trên 1 dòng.
 - T, S thỏa mãn ràng buộc: $1 \leq T \leq 10; 1 \leq \text{length}(S) \leq 10;$
- Output:**
- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2 AB ABC	AB BA ABC ACB BAC BCA CAB CBA

Chú ý : bài toán sinh hoán vị số từ 1 đến n:

Giải :

```
#include<bits/stdc++.h>
using namespace std;
string x[100];
string s;
bool chuaxet[100];

void out(int n){
    for(int i=1;i<=n;i++) cout<<x[i];
    cout<<" ";
}
```

```
void tryy(int i){
    for(int j=0;j<s.size();j++){
        if(chuaxet[j]){
            x[i+1]=s[j];
            chuaxet[j]=false;
            if(i==s.size()-1) out(s.size());
            else tryy(i+1);
            chuaxet[j]=true;
        }
    }
}

int main(){
    int t;
```

```
cin>>t;
while(t--){
    cin>>s;
    for(int i=0;i<s.size();i++) chuaxet[i]=true;
    tryy(0);

    cout<<endl;
}
return 0;
```

BÀI 9. DI CHUYỂN TRONG MÊ CUNG

Nội dung bài tập

Cho một mê cung bao gồm các khối được biểu diễn như một ma trận nhị phân A[N][N]. Một con chuột đi từ ô đầu tiên góc trái (A[0][0]) đến ô cuối cùng góc phải (A[N-1][N-1]) theo nguyên tắc:

- Down (D): Chuột được phép xuống dưới nếu ô dưới nó có giá trị 1.
- Right (R): Chuột được phép sang phải dưới nếu ô bên phải nó có giá trị 1.
- Left (L): Chuột được phép sang trái dưới nếu ô bên trái nó có giá trị 1.
- Up (U): Chuột được phép lên trên nếu ô trên nó có giá trị 1.

Hãy đưa ra tất cả các hành trình của con chuột trên mê cung. Đưa ra -1 nếu chuột không thể đi đến đích.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N là kích cỡ của mê cung; dòng tiếp theo đưa vào ma trận nhị phân A[N][N].
- T, N, A[i][j] thỏa mãn ràng buộc: $1 \leq T \leq 10; 2 \leq N \leq 8; 0 \leq A[i][j] \leq 1.$

Output:

- Đưa ra các xâu ký tự được sắp xếp, trong đó mỗi xâu là một đường đi của con chuột trong mê cung. In ra đáp án theo thứ tự từ điển. Đưa ra -1 nếu chuột không đi được đến đích.

Input	Output
3	DRDDRR
4	DDRDRR DRDDRR
1 0 0 0	DDRRURRDDDD DDRURRRDDDD DRDRURRDDDD DRRRR

1 1 0 1	}
0 1 0 0	check[x][y]=1;
0 1 1 1	if(x+1<n&& a[x+1][y]==1&&check[x+1][y]==0){
4	s+="D";
1 0 0 0	start(x+1,y);
1 1 0 1	s.erase(s.size()-1);
1 1 0 0	}
0 1 1 1	if(y-1>=0&& a[x][y-1]==1&&check[x][y-1]==0){
5	s+="L";
1 0 0 0 0	start(x,y-1);
1 1 1 1 1	s.erase(s.size()-1);
1 1 1 0 1	}
0 0 0 0 1	if(y+1<n&& a[x][y+1]==1&&check[x][y+1]==0){
0 0 0 0 1	s+="R";
	start(x,y+1);
	s.erase(s.size()-1);
	}

Chú Ý:

-str.erase(x, y); *// Xóa đi y ký tự bắt đầu tại vị trí x.*

str.erase(str.begin() + x); *// Xóa đi ký tự tại vị trí x.*

str.erase(str.begin() + x, str.begin() + y); *// Xóa đi các ký tự từ chỉ số x đến chỉ số y - 1*

-Chú ý: Với các hàm có kiểu trả về là void. Bạn dùng lệnh return; và không có giá trị phía sau. Lệnh return trong hàm đệ quy nó mang nghĩa quay lại những trạng thái được lưu trong stack trước đó.

-

Giải :

```
#include<bits/stdc++.h>
using namespace std;
int a[10][10],check[10][10]={ };
int n;
bool test;
string s;
void start(int x,int y){
    if(x==n-1&&y==n-1){
        cout<<s<<" ";
        test =1;
        return;
    }
```

```

    }

    if(x-1>=0&&a[x-1][y]==1&&check[x-1][y]==0){
        s+="U";
        start(x-1,y);
        s.erase(s.size()-1);
    }
    check[x][y]=0;
}

int main(){
    int t;
    cin>>t;
    for(int i=0;i<t;i++){
        s="";
        test=0;
        cin>>n;

        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                cin >>a[i][j]; check[i][j]=0;
            }
        }
    }
}
```



```

        }
    }

    if(a[0][0]==0) {cout<<-1<<endl;continue;}
    start(0,0);
    if(test==0) cout<<-1;
    cout<<endl;
}
return 0;
}

```

BÀI 10. TỔ HỢP SỐ CÓ TỔNG BẰNG X

Nội dung bài tập

Cho mảng $A[]$ gồm N số nguyên dương phân biệt và số X . Nhiệm vụ của bạn là tìm phép tổ hợp các số trong mảng $A[]$ có tổng bằng X . Các số trong mảng $A[]$ có thể được sử dụng nhiều lần. Mỗi tổ hợp các số của mảng $A[]$ được in ra theo thứ tự không giảm các số. Ví dụ với $A[] = \{2, 4, 6, 8\}$, $X = 8$ ta có các tổ hợp các số như sau:

[2, 2, 2, 2], [2, 2, 4], [2, 6], [4, 4], [8].

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là hai số N và X ; dòng tiếp theo đưa vào N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, X, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 10$; $1 \leq X, A[i] \leq 100$. $N \leq 20$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Mỗi đường tổ hợp được bao bởi cặp ký tự [,]. Đưa ra -1 nếu không có tổ hợp nào thỏa mãn yêu cầu bài toán.

Input	Output
1	
4 8	[2 2 2 2] [2 2 4] [2 6] [4 4] [8]
2 4 6 8	

Giải:

```

#include<bits/stdc++.h>
using namespace std;
int n,s;
int a[100],b[100];
bool ch;
void out(int x){
    cout<<"[";
    for(int i=1;i<=x;i++) cout<<b[i]<<" ";
    cout<<b[x-1]<<"]";//ko co dau cach
}
void go(int x,int sum,int cur){
    for(int i=cur;i<=n;i++){
        if(sum==s) {
            out(x); ch=1;
            break;
        }
        if(sum+a[i]<=s){
            b[x]=a[i];
            sum+=a[i];
            go(x+1,sum,i);
            sum-=a[i];
        }
    }
}
int main(){
    int t;
    cin>>t;
    for(int i=0;i<t;i++){
        cin>>n>>s;
        ch=0;
        for(int i=1;i<=n;i++) cin>>a[i];
        sort(a+1,a+n+1);
        go(1,0,1);
        if(ch==0)cout<<-1;
        cout<<endl;
    }
}

```

```

    return 0;
}

```

BÀI 11. DI CHUYỂN TRONG MA TRẬN

Nội dung bài tập

Cho ma trận $A[M][N]$. Nhiệm vụ của bạn là đưa ra tất cả các đường đi từ phần tử $A[0][0]$ đến phần tử $A[M-1][N-1]$. Bạn chỉ được phép dịch chuyển xuống dưới hoặc sang phải phần tử liền kề với vị trí hiện tại.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là hai số M, N tương ứng với số hàng và số cột của ma trận; dòng tiếp theo đưa vào các phần tử của ma trận $A[i][j]$; các số được viết cách nhau một vài khoảng trống.
- $T, M, N, A[i][j]$ thỏa mãn ràng buộc: $1 \leq T \leq 10; 1 \leq M, N, A[i][j] \leq 100$.

Output:

- Đưa ra số cách di chuyển của mỗi test theo từng dòng.
- Giải thích test 1: Có 3 cách di chuyển là $[1 \ 4 \ 5 \ 6], [1 \ 2 \ 5 \ 6]$ và $[1 \ 2 \ 3 \ 6]$.

Input	Output
2	
2 3	
1 2 3	3
4 5 6	2
2 2	
1 2	
3 4	

Xem ảnh vũ thành công

BÀI 12. SỐ NGUYÊN TỐ

cho ba số N, P, S . Trong đó, P là một số nguyên tố. Nhiệm vụ của bạn là đưa ra tất cả N số nguyên tố tính từ P có tổng bằng S . Ví dụ với $S = 28, P = 7, N = 2$ ta có kết quả $11 + 17 = 28$. Với $N = 3, P = 2, S = 23$ ta có kết quả: $\{3, 7, 13\}, \{5, 7, 11\}$

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ ba số S, P, N được viết trên một dòng.
- S, P, N thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq N \leq 10; 2 \leq S, P \leq 200$.

Output:

- Với mỗi test, dòng đầu tiên in ra số lượng đáp án tìm được. Mỗi dòng tiếp theo in ra kết quả tìm được theo thứ tự từ điển.

Input	Output
1	1
2	11 17
2 7 28	2
3 2 23	3 7 13
	5 7 11

```

#include <bits/stdc++.h>
using namespace std;
int a[100];
int n,s;
bool check;
void Try(int vt,int pos,int sum)
{
    if(check==false)
    {
        if(sum>s) return;
        else if(sum==s)
        {
            cout<<pos<<endl;
            check=true;
        }
        for(int i=vt;i>=0;i--)
        {
            Try(i-1,pos+1,sum+a[i]);
        }
    }
}
int main(){

```

```

int t;
cin>>t;
while(t-->0)
{
    cin>>n>>s;
    check=false;
    for(int i=0;i<n;i++) cin>>a[i];
    sort(a,a+n);
    Try(n-1,0,0);
    if(check==false) cout<<"-1"<<endl;
}
return 0;
}

```

BÀI 13. SẮP XẾP QUÂN HẬU

Nội dung bài tập

Cho một bàn cờ 8 x 8, mỗi ô có một giá trị $A[i][j]$ nhất định ($0 \leq A[i][j] \leq 100$), tương ứng với điểm số đạt được nếu như bạn đặt một quân cờ vào đó. Nhiệm vụ của bạn là đặt 8 quân hậu lên bàn cờ, sao cho không có 2 quân nào ăn nhau, và số điểm đạt được là lớn nhất.

Input: Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test gồm 8 dòng, mỗi dòng 8 số nguyên mô tả bàn cờ.

Output: Với mỗi test, in ra đáp án trên một dòng.

Ví dụ:

Input	Output
<pre> 1 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 48 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 </pre>	<pre> 260 </pre>

Note : -count được xác định trong cả mã của bạn và bởi Thư viện Chuẩn (trong không gian tên std). Việc bạn sử dụng using namespace std; để kéo toàn bộ không gian tên Chuẩn vào không gian tên chung sẽ tạo ra sự mơ hồ.

Giải :

```

#include<bits/stdc++.h>
using namespace std;

```

```

int a[10][10];
bool cot[20]={0}, cheo1[20]={0}, cheo2[20]={0};
int temp=0;

```

```

void timhau(int i,int sum){
    for (int j=1;j<=8;j++){
        if(cot[j]==0&& cheo1[j-i+8]==0&&cheo2[j+i-1]==0){
            sum+=a[i][j];
            cot[j]=1;
            cheo1[j-i+8]=1;//j-i not i-j
            cheo2[j+i-1]=1;
            if(i==8){
                if(temp<sum) temp=sum;
            } else timhau(i+1,sum);
            sum-=a[i][j];
            cot[j]=0;
            cheo1[j-i+8]=0;
            cheo2[j+i-1]=0;
        }
    }
}

```

```

int main(){
    int t;
    cin>>t;
    while(t--){
        temp=0;
        for(int i=1;i<=8;i++){
            for(int j=1;j<=8;j++){

```

```

        cin >> a[i][j];
    }
}
timhau(1,0);
cout<<temp<<endl;
}
return 0;
}

```

BÀI 14. TẬP HỢP

Nội dung bài tập

Xét tất cả các tập hợp các số nguyên dương có các phần tử khác nhau và không lớn hơn số n cho trước. Nhiệm vụ của bạn là hãy đếm xem có tất cả bao nhiêu tập hợp có số lượng phần tử bằng k và tổng của tất cả các phần tử trong tập hợp bằng s ?

Các tập hợp là hoán vị của nhau chỉ được tính là một.

Ví dụ với $n = 9, k = 3, s = 23$, $\{6, 8, 9\}$ là tập hợp duy nhất thỏa mãn.

Input: Gồm nhiều bộ test (không quá 100 test).

Mỗi bộ test gồm 3 số nguyên n, k, s với $1 \leq n \leq 20, 1 \leq k \leq 10$ và $1 \leq s \leq 155$. Input kết thúc bởi 3 số 0.

Output: Với mỗi test in ra số lượng các tập hợp thỏa mãn điều kiện đề bài.

Ví dụ:

Input	Output
9 3 23	1
9 3 22	2
10 3 28	0
16 10 107	20
20 8 102	1542
20 10 105	5448
20 10 155	1
3 4 3	0
4 2 11	0
0 0 0	

```
#include<bits/stdc++.h>
```

```

using namespace std;
int n,k,s;
int a[11];
bool ch[21]={ };
int c;
void out(){
    int sum=0; //note
    for(int i=1;i<=k;i++){
        sum+=a[i]; //note
    }
    if (sum==s) c++; //out
}

void start(int i){
    for(int j=a[i-1]+1;j<=n-k+i;j++){
        if(ch[j]==0){
            a[i]=j;
            ch[j]==1;
            if(i==k) out();
            else start(i+1);
            ch[j]=0;
        }
    }
}

int main(){
    while(1){
        c=0;
        cin>>n>>k>>s;
        if(n==0&&k==0&&s==0) break;
        a[0]=0;
        start(1);
        cout<<c<<endl;
    }
    return 0;
}

```

BÀI 15. MÁY ATM

Nội dung bài tập

Một máy ATM hiện có n ($n \leq 30$) tờ tiền có giá trị $t[1], t[2], \dots, t[n]$. Hãy tìm cách trả ít tờ nhất với số tiền đúng bằng S (các tờ tiền có giá trị bất kỳ và có thể bằng nhau, mỗi tờ tiền chỉ được dùng một lần).

Input: Dòng đầu tiên ghi số bộ test T ($T < 10$). Mỗi bộ test gồm 2 số nguyên n và S ($S \leq 10^9$). Dòng thứ hai chứa n số nguyên $t[1], t[2], \dots, t[n]$ ($t[i] \leq 10^9$).

Output: Với mỗi bộ test ghi ra số tờ tiền ít nhất phải trả.

Nếu không thể tìm được kết quả, in ra -1.

Ví dụ

Input	Output
1 3 5 1 4 5	1

BÀI 16. LŨY THỪA

Nội dung bài tập

Cho số nguyên dương N và K . Hãy tính N^K modulo 10^9+7 .

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test gồm 1 số nguyên N và K ($1 \leq N \leq 1000, 1 \leq K \leq 10^9$).

Output:

Với mỗi test, in ra đáp án trên một dòng.

Ví dụ:

Input:	Output
2 2 3 4 2	8 16

C1:

```
#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007
using namespace std;
ll luythua(int a,ll n){
```

```
if (n==0) return 1;
ll t=luythua(a,n/2%mod);
if(n%2==0) return t*t%mod;
else return a*t%mod*t%mod;
```

```
}
int main(){
    int t;
    cin>>t;
    while(t--){
        int a;
        ll n;
        cin>>a>>n;
        cout<<luythua(a,n)<<endl;
    }
    return 0;
```

```
}
C2:
#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007
using namespace std;
ll power(ll a, ll b)
{
    if (b == 1)
        return a % mod;
    ll tmp = power(a, b / 2);
    tmp = tmp * tmp % mod;
    if (b % 2 == 1)
        return tmp * a % mod;
    return tmp;
}
```

```
int main(){
    int t;
    cin>>t;
    while(t--){
        int a;
```

```

    ll n;
    cin>>a>>n;
    cout<<power(a,n)<<endl;
    }
    return 0;
}

```

BÀI 17. GẤP ĐÔI DÃY SỐ

Nội dung bài tập

Một dãy số tự nhiên bắt đầu bởi con số 1 và được thực hiện N-1 phép biến đổi “gấp đôi” dãy số như sau:

Với dãy số A hiện tại, dãy số mới có dạng A, x, A trong đó x là số tự nhiên bé nhất chưa xuất hiện trong A.

Ví dụ với 2 bước biến đổi, ta có [1] à [1 2 1] à [1 2 1 3 1 2 1].

Các bạn hãy xác định số thứ K trong dãy số cuối cùng là bao nhiêu?

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test gồm số nguyên dương N và K ($1 \leq N \leq 50, 1 \leq K \leq 2^N - 1$).

Output:

Với mỗi test, in ra đáp án trên một dòng.

Ví dụ:

Input	Output
2	2
3 2	4
4 8	

Giải thích test 1: Dãy số thu được là [1, 2, 1, 3, 1, 2, 1].

Giải thích test 2: Dãy số thu được là [1, 2, 1, 3, 1, 2, 1, 4, 1, 2, 1, 3, 1, 2, 1].

Giải:

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n;
```

```
long long k,s,e;
```

```
void huyinitwsolve(){
```

```
    s=0;
```

```
    e=pow(2,n);
```

```

while(n){
    long long mid=(s+e)/2;
    if(k==mid) {
        cout<<n<<endl;break;
    }
    if(mid<k) s=mid;
    else e=mid;
    n--;
}
}

```

```

int main(){
    int t;
    cin>>t;
    while(t--){
        cin>>n>>k;
        if(k%2==1) cout<<1<<endl;
        else huyinitwsolve();

    }
    return 0;
}

```

Cách 2 :

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n;
```

```
long long k;
```

```
int devide(int n, long long k){
```

```
    long long x=pow(2,n-1);//so chinh giua
```

```
    if(x==k) return n;
```

```
    if(x>k) return devide(n-1,k);//k o nua trai
```

```
    else return devide(n-1,k-x);//k o nua phai
```

```
}
```

```
int main (){
```

```
    int t;
```

```
cin>>t;
while(t--){
    cin>>n>>k;
    cout<<devide(n,k)<<endl;;
}
}
```

BÀI 18. DÃY XÂU FIBONACI

Nội dung bài tập

Một dãy xâu ký tự G chỉ bao gồm các chữ cái A và B được gọi là dãy xâu Fibonacci nếu thỏa mãn tính chất: $G(1) = A; G(2) = B; G(n) = G(n-2)+G(n-1)$. Với phép cộng (+) là phép nối hai xâu với nhau. Bài toán đặt ra là tìm ký tự ở vị trí thứ i (tính từ 1) của xâu Fibonacci thứ n.

Dữ liệu vào: Dòng 1 ghi số bộ test. Mỗi bộ test ghi trên một dòng 2 số nguyên N và i ($1 \leq N \leq 93$). Số i đảm bảo trong phạm vi của xâu G(N) và không quá 18 chữ số. **Kết quả:** Ghi ra màn hình kết quả tương ứng với từng bộ test.

Input	Output
2	A
6 4	B
8 19	

```
#include<bits/stdc++.h>
using namespace std;
int n;
long long k;
long long a[100];
char tinh(int n,long long k){
    if(n==1) return 'A';
    if(n==2) return 'B';
    if(k<=a[n-2]) return tinh(n-2,k);
                                return tinh(n-1,k-a[n-2]);
}
int main(){
    int t;
    cin>>t;
    a[0]=0,a[1]=1;
    for(int i=2;i<=92;i++) a[i]=a[i-1]+a[i-2];
```

```
while(t--){
    cin>>n>>k;
    cout<<tinh(n,k)<<endl;
}
}
```

BÀI 19. SỐ FIBONACCI THỨ N

Nội dung bài tập

Dãy số Fibonacci được xác định bằng công thức như sau:

$F[0] = 0, F[1] = 1;$

$F[n] = F[n-1] + F[n-2]$ với mọi $n \geq 2$.

Các phần tử đầu tiên của dãy số là 0, 1, 1, 2, 3, 5, 8, ...

Nhiệm vụ của bạn là hãy xác định số Fibonacci thứ n. Do đáp số có thể rất lớn, in ra kết quả theo modulo 10^9+7 .

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 1000$).

Mỗi test bắt gồm một số nguyên N ($1 \leq N \leq 10^9$).

Output:

Với mỗi test, in ra đáp án trên một dòng.

Ví dụ:

Input:	Output
3	
2	1
6	8
20	6765

```
#include<stdio.h>
int M=1e9+7;
struct Matran{
    long long f[2][2];
};
Matran nhan(Matran A, Matran B){
    Matran C; int i,j,k;
    for(i=0; i<=1; i++){
        for(j=0; j<=1; j++){
            C.f[i][j]=0;
            for(k=0; k<=1; k++){
```

```

        C.f[i][j]=(C.f[i][j]+(A.f[i][k]*B.f[k][j])%M)%M;
    }
    return C;
}

Matran powerM(Matran A, int n){
    if(n == 1) return A;
    Matran x = powerM(A,n/2);
    if(n%2==0)return nhan(x,x);
    return nhan(A,nhan(x,x));
}

main(){
    int t,n;
    Matran A; scanf("%d",&t);
    while(t--){
        scanf("%lld",&n);
        A.f[0][0] = 0; A.f[0][1] = 1;
        A.f[1][0] = 1; A.f[1][1] = 1;
        Matran KQ = powerM(A,n-1);
        printf("%lld\n",KQ.f[1][1]);
    }
}

```

BÀI 20. DÂY CON LIÊN TIẾP CÓ TỔNG LỚN NHẤT

Chưa làm

Nội dung bài tập

Cho mảng $A[]$ gồm N số có cả các số âm và số dương. Nhiệm vụ của bạn là tìm mảng con liên tục có tổng lớn nhất của mảng. Ví dụ với mảng $A[] = \{-2, -5, 6, -2, -3, 1, 5, -6\}$ ta có kết quả là 7 tương ứng với dãy con $\{6, -2, -3, 1, 5\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào hai số N tương ứng với số phần tử của mảng; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.

- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq N \leq 100; -100 \leq A[i] \leq 100$.

Output:

- Đưa ra tổng con liên tục lớn nhất của mỗi test theo từng dòng.

Ví dụ:

Input	Output
1 8 -2 -5 6 -2 -3 1 5 -6	7

Giải :

```

#include<iostream>
#define ll long long
using namespace std;
const int MAX = 111;
int main(){
    int test; cin >> test;
    while(test--){
        int n; cin >> n;
        int A[MAX], Q[MAX], ans = 0;
        for(int i=0; i<n; i++) cin >> A[i];
        Q[0] = (A[0] > 0)? A[0] : 0;
        for(int i=1; i<n; i++){
            Q[i] = (Q[i-1] > 0)? A[i] + Q[i-1] : A[i];
            ans = (Q[i] > ans)? Q[i] : ans;
        }
        cout << ans << endl;
    }
}

```

tham lam , chia và trị

BÀI 1. LŨY THỪA MA TRẬN

Nội dung bài tập

Cho ma trận vuông A kích thước $N \times N$. Nhiệm vụ của bạn là hãy tính ma trận $X = A^K$ với K là số nguyên cho trước. Đáp số có thể rất lớn, hãy in ra kết quả theo modulo 109+7.

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 100$).
 Mỗi test bắt gồm một số nguyên N và K ($1 \leq N \leq 10, 1 \leq K \leq 109$) là kích thước của ma trận và số mũ.
 Output:
 Với mỗi test, in ra kết quả của ma trận X.
 Ví dụ:

Input:	Output
2	
2 5	8 5
1 1	5 3
1 0	597240088 35500972 473761863
3 1000000000	781257150 154135232 527013321
1 2 3	965274212 272769492 580264779
4 5 6	
7 8 9	

```
#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007 //them 1 so 0 la sai
using namespace std;
int n;
ll k;

struct matrix{
    long long pos[15][15];
};
matrix multi(matrix a,matrix b){
    matrix temp;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            temp.pos[i][j]=0;
            for(int k=0;k<n;k++){
                temp.pos[i][j] =(
temp.pos[i][j]+a.pos[i][k]*b.pos[k][j]%mod) %mod;//%mod
            }
        }
    }
```

```
    }
    return temp;//return
}
matrix powermod(matrix a,ll k){
    if(k==1) return a;
    if(k%2==0) return powermod(multi(a,a) ,k/2);
    return multi(a, powermod(multi(a,a) ,k/2) );
}

void huyinitwsolve(){
    int n; ll k; khai bao toan cuc de su dung
    matrix a;
    cin>>n>>k;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            cin>>a.pos[i][j];
        }
    }
    matrix b=powermod(a,k);
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            cout<<b.pos[i][j]<<" ";
        }
        cout<<endl;
    }
}

int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}
```

BÀI 2. LŨY THỪA ĐẢO

Nội dung bài tập
Cho mảng số N. Ta gọi số đảo của N là R. Hãy tìm lũy thừa R của N. Đưa ra kết quả của bài toán dưới dạng modulo với 109 + 7.
Input:
Dòng đầu tiên đưa vào số lượng bộ test T.
Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm là số N được ghi trên một dòng.
T, N thỏa mãn ràng buộc: 1≤T≤100; 1≤N≤1010.
Output:
Đưa ra kết quả mỗi test theo từng dòng.
Ví dụ:

Input	Output
2	
2	4
12	864354781

```
#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007
using namespace std;
ll luythua(ll a,ll b){
    if(b==1) return a%mod;
    ll temp= luythua( a, b/2 );
    /* temp=temp*temp%mod;
    if(b%2==0) return temp;
    return temp*a%mod; //chu y khong phai a%mod*temp*/
    if(b%2==0) return temp*temp%mod;
    return temp*temp%mod*a%mod;
}

int main(){
    int t;
    cin>>t;
    while(t--){
        long long n;
        cin>>n;
        long long m=n,r=0;
```

```
while(m>0){
    r=r*10+m%10;
    m/=10;
}
cout<<luythua(n,r)<<endl;
}
```

BÀI 3. TÍCH HAI SỐ NHỊ PHÂN

Nội dung bài tập
Cho hai xâu nhị phân biểu diễn hai số. Nhiệm vụ của bạn là đưa ra tích của hai số. Ví dụ với xâu S1="1100" và S2="1010" ta sẽ có kết quả là 120.
Input:
Dòng đầu tiên đưa vào số lượng bộ test T.
Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 hai xâu nhị phân S1, S2 được viết trên một dòng.
T, S1, S2 thỏa mãn ràng buộc: 1≤T≤100; 1≤ length(S1), length(S2)≤30.
Output:
Đưa ra tích của mỗi test theo từng dòng.
Ví dụ:

Input	Output
2	
1100 01	12
01 01	1

```
#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007
using namespace std;
string s1,s2;
ll he10(string s){
    ll sum=0;
    for(int i=0;i<s.length();i++){
        sum*=2;
        sum+=(s[i]=='1'); //note '1' khong phai 1
    }
}
```

```

        return sum;//chu y return
    }
    void huyinitwsolve(){
        cin>>s1>>s2;
        cout<<he10(s1)*he10(s2)<<endl;
    }

    int main(){
        int t;
        cin>>t;
        while(t--){
            huyinitwsolve();
        }
    }

```

BÀI 4. SẮP XẾP CÔNG VIỆC

Nội dung bài tập

Cho hệ gồm N hành động. Mỗi hành động được biểu diễn như một bộ đôi <Si, Fi> tương ứng với thời gian bắt đầu và thời gian kết thúc của mỗi hành động. Hãy tìm phương án thực hiện nhiều nhất các hành động được thực hiện bởi một máy hoặc một người sao cho hệ không xảy ra mâu thuẫn.

Input:

Dòng đầu tiên đưa vào số lượng bộ test T.

Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào số lượng hành động N; dòng tiếp theo đưa vào N số Si tương ứng với thời gian bắt đầu mỗi hành động; dòng cuối cùng đưa vào N số Fi tương ứng với thời gian kết thúc mỗi hành động; các số được viết cách nhau một vài khoảng trống.

T, N, Si, Fi thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $Fi, Si \leq 1000$.

Output:

Đưa số lượng lớn nhất các hành động có thể được thực thi bởi một máy hoặc một người.

Ví dụ:

Input	Output
1 6 1 3 0 5 8 5	4

```

2 4 6 7 9 9

```

```

#include<bits/stdc++.h>
using namespace std;
int n,s[1005],f[1005];

void huyinitwsolve(){
    cin>>n;
    for(int i=0;i<n;i++) cin>>s[i];
    for(int i=0;i<n;i++) cin>>f[i];
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(f[i]>f[j]){
                swap(f[i],f[j]);
                swap(s[i],s[j]);
            }
        }
    }
    int dem=1, j=0;
    for(int i=1;i<n;i++){
        if(s[i]>=f[j]) {/=
            dem++;j=i;
        }
    }
    cout<<dem<<endl;
}

int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}

```

BÀI 5. NÓI DÂY

Nội dung bài tập

Cho N sợi dây với độ dài khác nhau được lưu trong mảng A[]. Nhiệm vụ của bạn là nối N sợi dây thành một sợi sao cho tổng chi phí nối dây là nhỏ nhất. Biết chi phí nối sợi dây thứ i và sợi dây thứ j là tổng độ dài hai sợi dây A[i] và A[j].

Input:

Dòng đầu tiên đưa vào số lượng bộ test T.

Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào số lượng sợi dây N; dòng tiếp theo đưa vào N số A[i] là độ dài của các sợi dây; các số được viết cách nhau một vài khoảng trống.

T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 106$; $0 \leq A[i] \leq 106$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
4	
4 3 2 6	29
5	62
4 2 7 6 9	

```
#include<bits/stdc++.h>
using namespace std;
```

```
int n;
long long sum;
//priority_queue <int, vector<int>,greater<int> > q; => ko dk khai bao ben ngoai
void huyinitwsolve(){
    sum=0;
    priority_queue <int, vector<int>,greater<int> > q;
    cin>>n;
    for(int i=0;i<n;i++){
        int x;
        cin>>x;
        q.push(x);
    }
```

```
while(q.size(>1){
    int first=q.top();
    q.pop();
    int second=q.top();
    q.pop();
    int temp=first+second;
    sum+=temp;
    q.push(temp);
}
cout<<sum<<endl;
```

```
}
int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}
```

BÀI 6. SẮP ĐẶT XÂU KÝ TỰ

Nội dung bài tập

Cho chuỗi ký tự S bao gồm các ký tự in thường và số D. Nhiệm vụ của bạn là kiểm tra xem ta có thể sắp đặt lại các ký tự trong S để tất cả các ký tự giống nhau đều có khoảng cách là D hay không? Đưa ra 1 nếu có thể sắp đặt lại các ký tự trong S thỏa mãn yêu cầu bài toán, ngược lại đưa ra -1.

Input:

Dòng đầu tiên đưa vào số lượng bộ test T.

Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số D; dòng tiếp theo là chuỗi S.

T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 10000$; $1 \leq D \leq 100$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	1
2	-1

ABB	
2	
AAA	

```

Note:
#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007
using namespace std;
void huyinitwsolve(){
    int d;
    string s;
    cin>>d>>s;
    int a[100]={ },dem=0;
    for(int i=0;i<s.length();i++){
        a[s[i]-'A']++;
        dem=max(a[s[i]-'A'],dem);
    }
    if( (dem-1)*(d-1)>s.length()-dem ) cout<<-1<<endl;
    //dem-1=khoang trong;;
    //d-1 la ki tu trong can dien trong moi khoang trong
    else cout<<1<<endl;
}

int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}

```

BÀI 7. GIÁ TRỊ NHỎ NHẤT CỦA XÂU

Nội dung bài tập

Cho chuỗi ký tự S. Ta gọi giá trị của chuỗi S là tổng bình phương số lần xuất hiện mỗi ký tự trong S. Hãy tìm giá trị nhỏ nhất của chuỗi S sau khi thực hiện K lần loại bỏ ký tự.

Input:

Dòng đầu tiên đưa vào số lượng bộ test T.

Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là số K; phần thứ hai là một chuỗi ký tự S được viết trên một dòng.

T, S, K thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 10000$; $1 \leq K \leq 1000$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
2	
ABCCBC	6
2	2
AAAB	

```

#include<iostream>
#include<algorithm>
#include<queue>
using namespace std;
void huyinitwsolve(){
    int k;cin>>k;
    string s; cin>>s;
    int dem[100]={ };//khởi tạo =0
    priority_queue< int> q;
    for(int i=0;i<s.length();i++) dem[s[i]-65]++;//s[i]-65 =>chuyển chữ
    hoa sang số >=0
    for(int i=0;i<60;i++) if(dem[i]>=1) q.push(dem[i]);
    while(k--){
        int x=q.top();
        x--;
        q.pop();
        q.push(x);
    }
    long long sum=0;
    while(q.empty()==0){
        sum+=q.top()*q.top();
    }
}

```

```

        q.pop();
    }
    cout<<sum<<endl;
}
int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}

```

BÀI 8. SỐ MAY MẮN

Nội dung bài tập

Hoàng yêu thích các số may mắn. Ta biết rằng một số là số may mắn nếu biểu diễn thập phân của nó chỉ chứa các chữ số may mắn là 4 và 7. Ví dụ, các số 47, 744, 4 là số may mắn và 5, 17, 467 không phải. Hoàng muốn tìm số may mắn bé nhất có tổng các chữ số bằng n. Hãy giúp anh ấy.

Dữ liệu vào: Dòng đầu ghi số bộ test, mỗi bộ test có một dòng chứa số nguyên n ($1 \leq n \leq 106$) — tổng các chữ số của số may mắn cần tìm.

Kết quả: In ra trên 1 dòng số may mắn bé nhất, mà tổng các chữ số bằng n. Nếu không tồn tại số thỏa mãn, in ra -1.

Ví dụ:

Input	Output
2	47
11	-1
10	

```

#include<iostream>
#include<algorithm>
#include<queue>
using namespace std;
void huyinitwsolve(){
    int sum;cin>>sum;
    priority_queue< int,vector<int>, greater<int> > q;
    int time7=sum/7; for(int i=0;i<time7;i++) q.push(7);
    int time4=sum%7;
    while(time4%4!=0&&q.empty()==0){

```

```

        time4+=q.top();
        q.pop();
    }
    if(time4%4==0){
        for(int i=0;i<time4/4;i++) q.push(4);
        while(q.empty()==0){
            cout<<q.top(); q.pop();
        }
        cout<<endl;
    }
    else cout<<-1<<endl;
}
int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}

```

BÀI 9. SỐ NHỎ NHẤT CÓ N ƯỚC SỐ

Nội dung bài tập

Cho số nguyên dương N. Nhiệm vụ của bạn là tìm số K nhỏ nhất, sao cho K có đúng N ước. Input đảm bảo rằng đáp án không vượt quá 1018.

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test gồm 1 số nguyên N ($1 \leq N \leq 1000$).

Output: Với mỗi test, in ra đáp án trên một dòng.

Ví dụ:

Input	Output
2	6
4	12
6	

```
#include<iostream>
#include<algorithm>
using namespace std;
#define ll long long
ll A[] = {2,3,5,7,11,13,17,19,23,29,31,37,41,43}, ans, n, res, better;
void backTrack(ll i){
    ll j = 1;
    while(true){
        res *= A[i];
        if(res >= ans) break;
        ll tmp1 = res;
        ll tmp = better;
        better *= (j+1);
        if(better > n) break;
        else if(better == n) ans = min(ans, res);
        else backTrack(i+1);
        j++; better = tmp; res = tmp1;
    }
}
int main(){
    int test; cin >> test;
    while(test--){
        ans = 1e18; res = 1, better = 1;
        cin >> n;
        if(n == 1) cout << 1;
        else{
            backTrack(0);
            cout << ans;
        }
        cout << endl;
    }
}
```

BÀI 10. KÝ TỰ ĐẶC BIỆT

Nội dung bài tập

Cho một xâu s. Xâu F(s) được xác định bằng cách ghép xâu xâu s ban đầu với xâu s sau khi đã được quay vòng sang bên phải 1 kí tự (kí tự cuối cùng của s được chuyển lên đầu).

Thực hiện liên tiếp các bước cộng xâu như trên với xâu mới thu được, ta có được xâu X.

với .

Nhiệm vụ của bạn là hãy xác định kí tự thứ N trong xâu X là kí tự nào?

Input: Dòng đầu ghi số bộ test T ($T < 10$). Mỗi bộ test gồm một xâu s có độ dài không vượt quá 30 kí tự và số nguyên N ($1 \leq N \leq 1018$).

Output: Với mỗi bộ test ghi ra trên một dòng kí tự tìm được.

Ví dụ:

Input	Output
1	C
COW 8	

Giải thích test: COW à COWWCO à COWWCOOCOWWC. Kí tự thứ 8 là ‘C’.

```
#include<bits/stdc++.h>
```

```
#define ll long long
```

```
using namespace std;
```

```
string s;
```

```
ll k;
```

```
ll Try (ll n){ //ll ko phai void
```

```
    if(k<=s.length()) return k-1;
```

```
    if(k>n/2) k=k-n/2-1;
```

```
    if (k==0) k=n/2;
```

```
    return Try(n/2);
```

```
}
```

```
void huyinitwsolve(){
```

```
    cin>>s>>k;
```

```
    ll n=s.length(),n1=n;
```

```
    while (n<k) n*=2;
```

```
    ll res= Try(n);
```

```
    cout<<s[res]<<endl;
```

```

}
int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}

```

BÀI 11. NGƯỜI DU LỊCH

Nội dung bài tập

Cho n thành phố đánh số từ 1 đến n và các tuyến đường giao thông hai chiều giữa chúng, mạng lưới giao thông này được cho bởi mảng $C[1 \dots n, 1 \dots n]$ ở đây $C[i][j] = C[j][i]$ là chi phí đi đoạn đường trực tiếp từ thành phố i đến thành phố j .

Một người du lịch xuất phát từ thành phố 1, muốn đi thăm tất cả các thành phố còn lại mỗi thành phố đúng 1 lần và cuối cùng quay lại thành phố 1. Hãy chỉ ra chi phí ít nhất mà người đó phải bỏ ra.

Dữ liệu vào: Dòng đầu tiên là số nguyên n – số thành phố ($n \leq 15$); n dòng sau, mỗi dòng chứa n số nguyên thể hiện cho mảng 2 chiều C .

Kết quả: Chi phí mà người đó phải bỏ ra.

Ví dụ:

INPUT	OUTPUT
4 0 20 35 10 20 0 90 50 35 90 0 12 10 50 12 0	117

```

C1 : pmd
#include <bits/stdc++.h>
#define ll long long

```

```
using namespace std;
```

```
const int INF = 1e9 + 7;
```

HUY INIT

```
const int MAXN = 1e6 + 7;
const double EPS = 1e-8;
```

```

int main() {
    int n;
    cin >> n;
    int a[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> a[i][j];
        }
    }
    const int N = 1 << n;
    vector <vector <int>> > dp(N, vector <int> (n, INF));
    dp[1][0] = 0;
    for (int mask = 0; mask < N; mask++) {
        for (int v = 0; v < n; v++) {
            if (dp[mask][v] == INF) continue;
            for (int u = 0; u < n; u++) {
                if (mask & (1 << u)) continue;
                dp[mask | (1 << u)][u] = min(dp[mask | (1 << u)][u], dp[mask][v]
+ a[v][u]);
            }
        }
    }
    int res = INF;
    for (int i = 0; i < n; i++) {
        res = min(res, dp[N - 1][i] + a[i][0]);
    }
    cout << res;
}

```

BÀI 12. HỢP VÀ GIAO CỦA HAI MẢNG SỐ

Nội dung bài tập

Cho mảng A[] gồm n phần tử, mảng B[] gồm m phần tử khác nhau. Các phần tử của mảng A[] và B[] chưa được sắp xếp. Hãy tìm mảng hợp và giao được sắp giữa A[] và B[]. Ví dụ với A[] = { 7, 1, 5, 2, 3, 6}, B[]={3, 8, 6, 20, 7} ta có mảng hợp Union = { 1, 2, 3, 5, 6, 7, 8, 20}, mảng giao Intersection = {3, 6}.

Input:

Dòng đầu tiên đưa vào số lượng bộ test T.

Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m là số phần tử của mảng A[] và B[]; dòng tiếp theo là n số A[i] của mảng A [];dòng tiếp theo là m số B[i] của mảng B[]; các số được viết cách nhau một vài khoảng trống.

T, n, m, A[i], B[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m, A[i], B[i] \leq 105$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	
6 5	1 2 3 5 6 7 8 20
7 1 5 2 3 6	2 6
3 8 6 20 7	

```
#include<bits/stdc++.h>
#define ll long long
using namespace std;
int a[100005],b[100005],uni[100005],inter[100005];//note 10 mu 5
int m,n;
void huyinitwsolve(){
    cin>>m>>n;
    for(int i = 0 ; i <= 100000 ; i++) {
        uni[i]=0, inter[i] = 0;
    }
    for(int i=0;i<m;i++) {
        cin>>a[i];uni[a[i]]=1;
    }
    for(int i=0;i<n;i++) {
        cin>>b[i];inter[b[i]]=1;
    }
}
```

```
for(int i=0;i<=100000;i++) if(uni[i]==1||inter[i]==1 ) cout<<i<<" ";
cout<<endl;
for(int i=0;i<=100000;i++) if(uni[i]==1&&inter[i]==1 ) cout<<i<<" ";
cout<<endl;
}
```

```
int main(){
    int t;
    cin>>t;
    while(t--){
        huyinitwsolve();
    }
}
```

BÀI 13. TỔNG CẬP SỐ NGUYÊN TỐ.

Nội dung bài tập

Cho số tự nhiên N. Hãy tìm cặp số nguyên tố đầu tiên có tổng là N. Nếu không tồn tại cặp số nguyên tố có tổng bằng N, hãy đưa ra -1.

Input:

Dòng đầu tiên đưa vào số lượng bộ test T.

Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm là một số N được ghi trên một dòng.

T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 106$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	2 2
4	2 5
8	

```
#include<iostream>
#include<algorithm>
#include<cmath>
using namespace std;
/*
```

```

sang nguyen to.
*/
const int N = 1e6 + 5 ;
int n;
int isPrime[N];
void sieve(){
    for(int i = 0 ; i <= N ; i++) isPrime[i] = 0; // khoi tao cac phan tu ban
    dau bang 0
    isPrime[0] = isPrime[1] = 1;
    for ( int i = 2 ; i <= sqrt(N) ; i++){
        if ( isPrime[i] == 0){
            for(int j = i * i ; j <= N ; j +=i)
                isPrime[j] = 1;
        }
    }
    // so nguyen to la cac so duoc danh dau bang 0.
}
void input(){
    cin >> n;
}

void solve(){
    int flag = 0;
    for( int i = 2 ; i <= n/2 ; i++){
        //cout << i << ' ' << n - i << endl;
        if (isPrime[i] == 0 && isPrime[n-i] == 0 ){
            cout << i << ' ' << n - i << endl;
            flag = 1;
            break;
        }
    }
    if(!flag)
        cout << -1 << endl;
    //cout << endl;
}

int main(){

```

```

    int t = 1;
    sieve(); // sang nguyen to
    cin >> t;
    while(t--){
        input() ;
        solve();
    }
    return 0;
}
Cách 2:
#include <iostream>

using namespace std;
int a[500000]={ ....sinh 500k so snguyen to sau copy}
int snt(int n)
{
    if(n<2)return 0;
    if(n==2)return 1;
    if(n%2==0)return 0;
    for(int i=3;i<=n/2;i+=2)
        if(n%i==0)return 0;
    return 1;
}
int main()
{
    int t;
    cin >> t;
    while(t--){
        int n;
        cin >> n;
        for(int i=0;i<500000;i++){
            if(a[i]>=n){
                cout << -1;
                break;
            }

```

```

else if(snt(n-a[i])){
    int mini=(a[i]<n-a[i]?a[i]:n-a[i];
    cout << mini << " " << n-mini;
    break;
}
}
cout << endl;
}
return 0;
}

```

BÀI 14. MERGE SORT

Nội dung bài tập

Cho mảng $A[]$ gồm N phần tử chưa được sắp xếp. Nhiệm vụ của bạn là sắp xếp các phần tử của mảng $A[]$ theo thứ tự tăng dần bằng thuật toán Merge Sort.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N tương ứng với số phần tử của mảng $A[]$; phần thứ 2 là N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^6$.

Output:

- Đưa ra kết quả các test theo từng dòng.

Input	Output
2	
5	
4 1 3 9 7	1 3 4 7 9
10	1 2 3 4 5 6 7 8 9 10
10 9 8 7 6 5 4 3 2 1	

```

#include<bits/stdc++.h>
#define ll long long
#define mod 1000000007
using namespace std;

```

```

int main(){
    int t;
    cin>>t;
    int a[100000];
    while(t--){
        int n;
        cin>>n;
        for(int i=0;i<n;i++){
            cin>>a[i];
        }
        sort(a,a+n);
        for(int i=0;i<n;i++){
            cout<<a[i]<<" ";
        }
        cout<<endl;
    }
}

```

BÀI 15. BỔ SUNG PHẦN TỬ

Nội dung bài tập

Cho mảng $A[]$ gồm n số nguyên dương. Gọi L, R là max và min các phần tử của $A[]$. Nhiệm vụ của bạn là tìm số phần tử cần thiết cần thêm vào mảng để mảng có đầy đủ các số trong khoảng $[L, R]$. Ví dụ $A[] = \{5, 7, 9, 3, 6, 2\}$ ta nhận được kết quả là 2 tương ứng với các số còn thiếu là 4, 8.

Input:

Dòng đầu tiên đưa vào số lượng bộ test T .

Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n , tương ứng với số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$; các số được viết cách nhau một vài khoảng trống.

$T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n$, $A[i] \leq 103$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
5	0

4 5 3 8 6	
3	
2 1 3	

```
#include<bits/stdc++.h>
using namespace std;
void huyinitwsolve(){
    int n,count=0;
    cin>>n;
    int a[n];
    for(int i=0;i<n;i++) cin>>a[i];
    sort(a,a+n);
    for(int i=0;i<n-1;i++) if (a[i+1]!=a[i]) count++;
    cout<<a[n-1]-a[0]-count<<endl;
}
int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        huyinitwsolve();
    }
    return 0;
}
```